

DualManip: Agentic Dynamic Manipulation via Dual-Path Semantic Reasoning and Geometric Adaptation

Chengxi Li^{1,*}, Yan Di^{2,*}, Yingyue Li^{1,*}, Ruida Zhang¹, Mingyang Li^{3,*}, Xiangyang Ji^{1,†}

Abstract—Vision-language models (VLMs) enable open-vocabulary reasoning for robot manipulation, but their high inference latency limits responsiveness in dynamic scenes. Many scene changes, however, alter object geometry without invalidating task intent. We present DualManip, a dual-path framework that decouples infrequent semantic reasoning from responsive geometric adaptation. The semantic path decomposes the task and grounds task-relevant interactions, followed by a constraint-solving module for pose optimization. During execution, the geometric path continuously updates template-to-observation correspondences from live RGB-D observations via a shape-adaptive network. These correspondences transfer task-relevant grasp contacts across observations, enabling on-line grasp reconstruction under object motion and non-rigid deformation. The Information Interaction Module bridges the two paths by initializing task-relevant grasps from semantic grounding, validating geometric updates, and triggering semantic replanning upon update failures. Real-world evaluation spans six manipulation tasks covering non-rigid deformation, articulated reconfiguration, rigid motion, and high-precision assembly across three settings: static, single-change, and continuous dynamic. DualManip demonstrates superior manipulation robustness, particularly under continuous scene changes, while achieving geometric adaptation approximately 46× faster than agentic verification and semantic replanning. Our project page: <https://lichengxi1.github.io/Dualmanip>

I. INTRODUCTION

Robots operating in real-world environments must manipulate objects while the scene continues to evolve during perception, planning, and execution [1]–[3]. Objects may be displaced, remain in motion, or undergo substantial deformation, especially in household and human-centered settings involving garments, ropes, soft objects, and articulated structures [4]–[7]. Compared with static rigid-object manipulation, these scenarios require both rapid perception–action updates and the ability to accommodate time-varying, non-rigid geometry. Responsive manipulation of dynamic and deformable objects therefore remains an important challenge for general-purpose robotic systems.

End-to-end Vision-Language-Action (VLA) models and World-Action Models (WAMs) offer a promising paradigm for dynamic manipulation by learning closed-loop policies directly from large-scale robot data [2]–[4], [8]. However,

¹ Chengxi Li, Yingyue Li, Ruida Zhang and Xiangyang Ji are with the Department of Automation, Tsinghua University, China. {lichengx21, yingyue-21, zhangrd23}@mails.tsinghua.edu.cn, xyji@tsinghua.edu.cn

² Yan Di is with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), China. diyan@hit.edu.cn

³ Mingyang Li is with the Beijing Institute of Control Engineering, China. lmy_hit@163.com

[†] Corresponding author

* Equal contribution

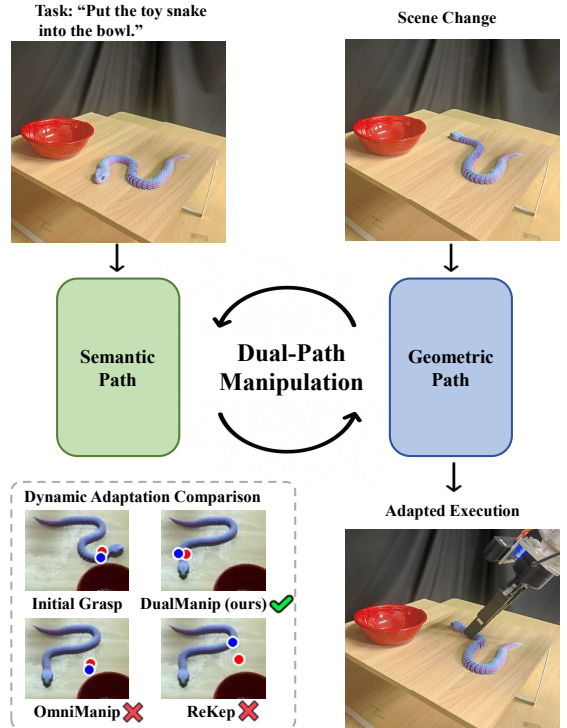


Fig. 1. DualManip decouples semantic reasoning from geometric grasp adaptation. As the toy snake deforms, the geometric path updates the grasp while preserving the task intent established by the semantic path. In contrast, ReKep and OmniManip fail to maintain a valid task-consistent grasp under the same non-rigid deformation.

these approaches typically require considerable training data tailored to specific tasks or robot embodiments and still incur the inference cost of large models. An alternative is a modular paradigm, in which pretrained Vision-Language Models (VLMs) provide open-vocabulary task reasoning while explicit geometric modules perform spatial grounding, motion generation, and low-level control [9]–[16]. By separating high-level reasoning from geometric execution, these systems provide interpretable interfaces and often generalize effectively to novel objects and instructions. Their ability to adapt to dynamic scenes, however, remains limited. Existing methods typically handle scene changes through geometric tracking, state verification, or episodic replanning [14]–[18]. Such mechanisms struggle in continuously evolving scenes: geometric methods (e.g., rigid pose or sparse point tracking) may degrade under non-rigid deformation, while agentic verification and replanning incur excessive latency.

The key observation is that scene geometry can change without invalidating task intent, as illustrated in Fig. 1. We

distinguish *geometric changes*, where task intent remains valid but prior scene geometry becomes outdated, from *semantic changes*, which alter task interpretation and require re-reasoning. Existing agentic systems lack an efficient mechanism to distinguish between them, causing frequent geometric changes to either trigger unnecessary high-latency VLM inference or leave robots operating on stale geometry.

To this end, we present DualManip, a *dual-path* architecture that separates deliberative semantic reasoning from responsive geometric adaptation. The *semantic path* performs VLM-based reasoning over the instruction and scene observation, producing a task-conditioned manipulation plan and grounding task-relevant object regions. The constraint-solving module then generates action constraints from these semantic outputs and optimizes for an executable pose. During execution, the *geometric path* continuously updates the scene geometry from live RGB-D observations and adapts the planned grasp without repeating semantic reasoning. Specifically, DualManip transfers task-relevant contact points from a pre-scanned object template to the current observation through point correspondence, and reconstructs the grasp pose from the transferred contacts under object motion and non-rigid deformation. The adapted grasp is then verified for reachability and collision safety before execution, while unreliable correspondence or infeasible geometry triggers a fallback to renewed semantic grounding and planning. By decoupling task semantics from time-varying geometry, DualManip preserves manipulation intent established by the VLM while enabling low-latency, online adaptation to continuously evolving scenes, including non-rigid deformation.

We evaluate DualManip on six real-world manipulation tasks spanning non-rigid deformation, articulated reconfiguration, rigid motion, and precision assembly. We further consider three evaluation settings: *static*, where the scene remains unchanged; *single-change*, where the object undergoes a single motion or deformation before remaining stationary; and *continuous dynamic*, where the object continues to move or deform during execution. Experimental results across diverse tasks confirm DualManip’s superior manipulation robustness, particularly under scene changes, while maintaining markedly lower adaptation latency than an agentic verification-and-replanning baseline.

The main contributions are summarized as follows:

- We propose DualManip, a dual-path framework that combines agentic semantic planning with responsive geometric adaptation, enabling intent-consistent action updates under dynamic scene variations.
- We develop a shape-adaptive correspondence network with geometry-aware learning that jointly models rigid motion and non-rigid deformation, enabling robust point-level matching across different object states.
- We introduce a task-conditioned contact-transfer and feasibility-routing mechanism for accurate grasp reconstruction, reliable feasibility validation, and selective fallback to semantic replanning.

II. RELATED WORK

A. Foundation Models for Robot Manipulation

End-to-end VLA policies directly map vision-language observations to robot actions, enabling increasingly general closed-loop manipulation [2], [4], [19]–[21]. WAMs further incorporate predictive world representations for long-horizon and dynamic execution [3], [8], [22]–[24]. Despite their strong capabilities, these approaches typically rely on considerable task- or embodiment-specific robot data and incur high training and inference costs. A complementary paradigm uses pretrained VLMs [25]–[27] for semantic reasoning together with explicit geometric or planning representations, including 3D value maps [9], part-level spatial constraints [10], [11], relational keypoint constraints [15], [28], object-centric primitives [16], semantic-to-action representations [12], [29], and task-and-motion planning [13]. DualManip follows this modular paradigm, targeting dynamic manipulation where object motion and non-rigid deformation require continuous geometric adaptation.

B. Agentic and Dynamic Manipulation

Recent agentic manipulation systems improve closed-loop execution through observation, verification, memory, tool orchestration, and recovery [14], [18], [30]–[34]. CLEA [18] and UniManip [14] maintain task and scene states for adaptive replanning, while AgentChord [30] and RoboReact [31] respond to execution disturbances through preplanned recovery or object-centric skill re-grounding. Other systems combine VLM- or LLM-based orchestration with VLA policies for low-level execution [32]–[34]. Despite improved closed-loop reasoning, adaptation typically occurs at the task or skill level and relies on discrete replanning. DualManip instead targets execution-time geometric changes, continuously adapting the planned grasp through a dedicated geometric path while preserving valid task semantics.

C. Geometric Tracking and Deformable Correspondence

Rigid object tracking models temporal motion with a single SE(3) transform [35], [36], while point tracking propagates correspondences in 2D or 3D space [37]–[39]. For deformable geometry, correspondence can be established through canonical deformation [40], [41], learned point matching [42], [43], or dense optical flow [44], [45]. These representations have supported rigid pose updates [16], [17], task-relevant point tracking [15], and deformable manipulation [5], [6]. In deformable manipulation, however, existing methods are often tailored to specific object categories or interaction patterns. Despite an object-level template prior, DualManip employs a unified template-to-observation correspondence mechanism for execution-time action transfer.

III. METHOD

A. Overview

Many scene changes alter object geometry without changing task intent. DualManip therefore separates semantic reasoning from geometric adaptation, as shown in Fig. 2.

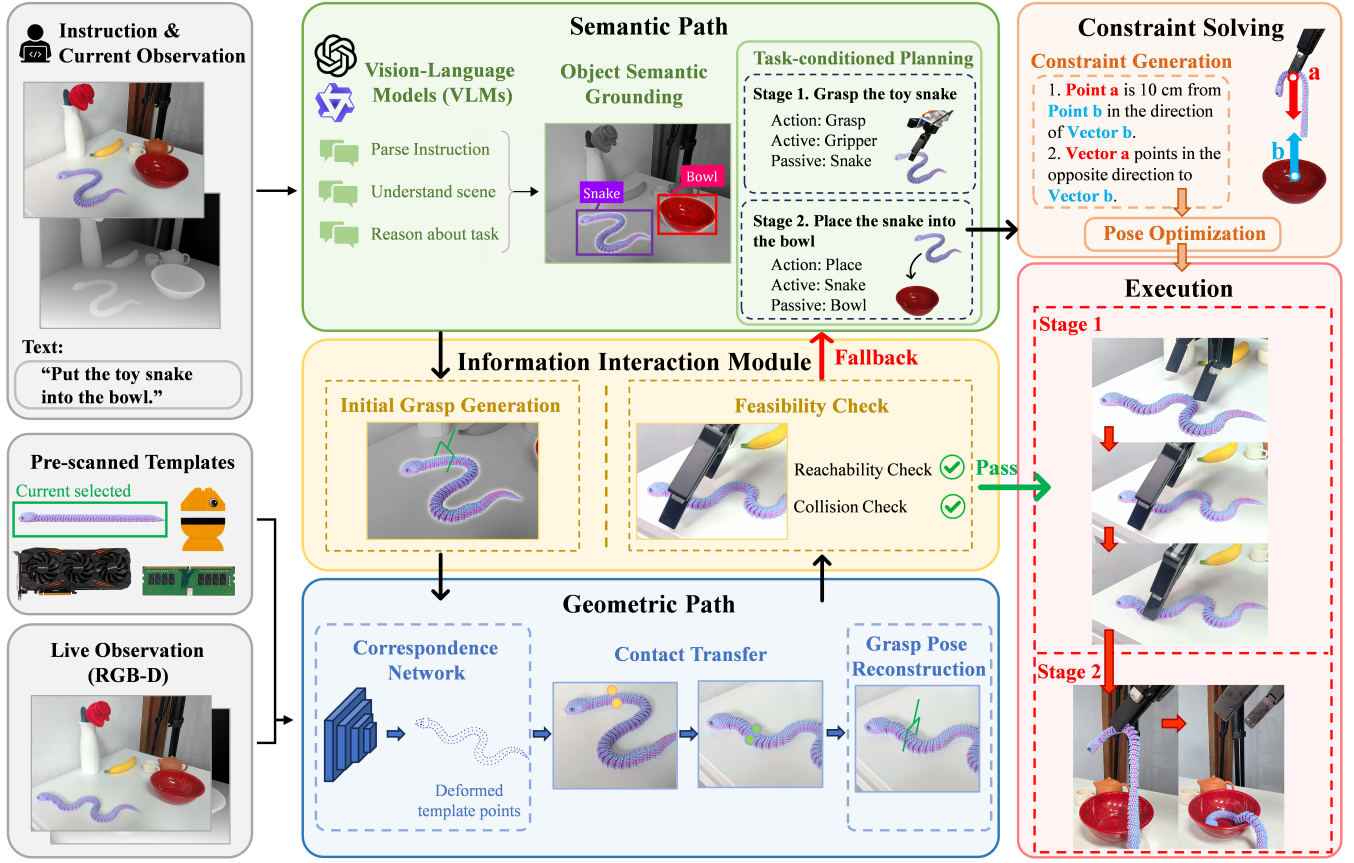


Fig. 2. Overview of DualManip. Given instruction and RGB-D observation, the semantic path establishes a task-conditioned plan, from which the information interaction module generates the initial grasp. During execution, the geometric path continuously updates the grasp from live RGB-D observations, executing feasible updates and falling back otherwise. For subsequent manipulation stages, task-conditioned spatial relations are formulated as constraints and optimized into executable poses.

The semantic path produces a staged plan and grounds task-relevant objects and regions. The geometric path transfers grasp contacts to the current geometry through template-to-observation correspondence, without repeating semantic reasoning. An information interaction module initializes the grasp, validates geometric updates, and invokes semantic replanning when adaptation is unreliable or infeasible. For subsequent manipulation stages, constraint solving converts task-conditioned spatial relations into executable poses. Together, these components allow DualManip to preserve valid task semantics while responding efficiently to object motion and non-rigid deformation.

B. Semantic Path and Constraint Solving

Given a manipulation instruction, the *Semantic Path* decomposes the task into multiple stages and identifies the task-relevant objects. For each stage, it grounds interaction locations and directions, while the constraint-solving module formulates spatial constraints, and optimizes for an executable end-effector pose.

Task Parsing. The manipulation task is first decomposed by a VLM [25], [26], yielding a sequence of stages $\mathcal{S} = \{S_i\}_{i=1}^n$. Following OmniManip [16], each stage is represented as $S_i = \{a_i, \mathcal{O}_i^a, \mathcal{O}_i^p\}$, where a_i denotes the action, and $\mathcal{O}_i^a$ and $\mathcal{O}_i^p$ denote the active and passive objects, respec-

tively. We then use Segment Anything Model 3 (SAM 3) [46] to localize these objects and produce their segmentation masks. The corresponding object point clouds $\mathbf{P}_i^a$ and $\mathbf{P}_i^p$ are recovered from the masks and depth observation.

Interaction Grounding. We characterize each interaction by a task-relevant location and direction. For stage S_i , the VLM identifies functional regions on both active and passive objects. Their descriptions are used to prompt SAM 3 on object-centric image crops, yielding local masks and the corresponding 3D interaction points $\mathbf{p}_i^a$ and $\mathbf{p}_i^p$. To obtain interaction directions, we employ Sofar [47] to infer task-conditioned semantic orientations $\mathbf{v}_i^o$ from the object point cloud $\mathbf{P}_i^o$ and region description l_i^o :

$$\mathbf{v}_i^o = \text{Sofar}(\mathbf{P}_i^o, l_i^o), \quad o \in \{a, p\}. \quad (1)$$

Constraint Generation. Utilizing these grounded interaction locations and directions, we formulate the desired interaction into explicit spatial constraints. Inspired by CoPa [10], we predefine a compact set of common geometric relations, encompassing point distances and directional alignments (e.g., perpendicularity and collinearity). We overlay the grounded points and directional vectors onto the scene image and query the VLM with the annotated image, stage intent, and predefined relation set to obtain the stage-specific

constraint set

$$\mathcal{C}_i = \{C_{i,k}\}_{k=1}^{m_i}, \quad (2)$$

where m_i is the number of constraints for stage i .

Pose Optimization. Each constraint $C_{i,k}$ is converted into a geometric violation cost $E_{i,k}(\mathbf{T})$ over the end-effector pose $\mathbf{T} \in \text{SE}(3)$. The target pose is obtained by minimizing the total constraint violation:

$$\mathbf{T}_i^* = \arg \min_{\mathbf{T} \in \text{SE}(3)} \sum_{k=1}^{m_i} \lambda_{i,k} E_{i,k}(\mathbf{T}), \quad (3)$$

where $\lambda_{i,k}$ controls the contribution of each constraint.

C. Geometric Path for Dynamic Adaptation

The *Geometric Path* enables low-latency grasp adaptation by maintaining point-level correspondences between a pre-scanned object template and live RGB-D observations, through which task-relevant grasp contacts are transferred to the current geometry. Unlike rigid pose or sparse point tracking, this template-based correspondence provides more robust handling of both object motion and non-rigid deformation.

Template Construction. For each manipulated object, we capture multi-view images and reconstruct a 3D template via Hunyuan3D [48], which is sampled into a reference point cloud $\mathbf{P}^{\text{ref}} \in \mathbb{R}^{M \times 3}$ as a persistent geometric reference. At time t , the current object point cloud $\mathbf{P}_t \in \mathbb{R}^{N_t \times 3}$ is extracted from the latest RGB-D frame, where M and N_t denote the numbers of template and observed points, respectively. Both $\mathbf{P}^{\text{ref}}$ and $\mathbf{P}_t$ are jointly processed by a correspondence network for point-level matching.

Shape-Adaptive Point Correspondence. To overcome severe geometric discrepancies caused by object motion and non-rigid deformation, inspired by [49], [50], we propose a shape-adaptive correspondence network that dynamically aligns dense points across disparate object states. Given the reference point cloud $\mathbf{P}^{\text{ref}}$ and the current observation $\mathbf{P}_t$, let $\mathbf{F}_t^{\text{ref}} \in \mathbb{R}^{M \times d}$ and $\mathbf{F}_t \in \mathbb{R}^{N_t \times d}$ denote their learned point-level features for correspondence estimation, where d is the feature dimension. After point-wise ℓ_2 normalization, their pairwise similarities are normalized through Sinkhorn iterations [51] to obtain a soft assignment matrix

$$\mathbf{A}_t = \text{Sinkhorn} \left(\frac{\mathbf{F}_t (\mathbf{F}_t^{\text{ref}})^\top}{\tau} \right) \in \mathbb{R}^{N_t \times M}, \quad (4)$$

where τ controls matching sharpness and each row of $\mathbf{A}_t$ specifies a correspondence distribution over the reference template. Rather than learning the assignment independently, we jointly predict a canonical deformation field $\Delta \mathbf{P}_t^{\text{ref}} \in \mathbb{R}^{M \times 3}$ and a global rigid transformation $(\mathbf{R}_t, \boldsymbol{\tau}_t)$. The reference point cloud is accordingly adapted to the current object geometry as

$$\hat{\mathbf{P}}_t^{\text{ref}} = (\mathbf{P}^{\text{ref}} + \Delta \mathbf{P}_t^{\text{ref}}) \mathbf{R}_t^\top + \boldsymbol{\tau}_t. \quad (5)$$

The soft assignment then associates each observed point with the adapted reference geometry:

$$\mathbf{X}_t = \mathbf{A}_t \hat{\mathbf{P}}_t^{\text{ref}}. \quad (6)$$

By coupling point assignment with the predicted deformation and rigid transformation, correspondences are structurally constrained by underlying geometry rather than learned from feature similarity alone. This geometric consistency underpins the correspondence losses introduced next.

Correspondence Learning. To promote motion- and deformation-invariant point correspondences, we train the correspondence network on paired RGB-D observations (s, t) using shared weights. For each pair, pseudo-ground-truth pixel correspondences $\mathcal{M}_{st} = \{(i, j)\}$ are generated via MARCO [52]. Specifically, we map each observation point to the undeformed canonical template as $\mathbf{Z}_r = \mathbf{A}_r \mathbf{P}^{\text{ref}} \in \mathbb{R}^{N_r \times 3}$, $r \in \{s, t\}$, and minimize

$$\mathcal{L}_{\text{pair}} = \frac{1}{|\mathcal{M}_{st}|} \sum_{(i,j) \in \mathcal{M}_{st}} \rho(\mathbf{z}_s^i, \mathbf{z}_t^j), \quad (7)$$

where $\rho(\cdot)$ denotes the SmoothL1 loss [53]. This objective encourages matched pixels to align with the same canonical template region, regardless of their current 3D positions.

For each observation r , we further impose a geometric consistency objective

$$\mathcal{L}_{\text{geo}} = \frac{\lambda_{\text{cd}}}{2} \sum_{r \in \{s,t\}} \mathcal{L}_{\text{cd}}(\mathbf{X}_r, \hat{\mathbf{P}}_r^{\text{ref}}) + \frac{\lambda_{3\text{D}}}{2} \sum_{r \in \{s,t\}} \rho(\mathbf{X}_r, \mathbf{P}_r), \quad (8)$$

where $\mathcal{L}_{\text{cd}}$ represents bidirectional Chamfer distance [54] that enforces template coverage, while the second term aligns template-mediated coordinates with observed 3D points.

Projection supervision is structured as

$$\mathcal{L}_{\text{proj}} = \lambda_{\text{kp}} \mathcal{L}_{\text{kp}} + \lambda_{\text{rep}} \mathcal{L}_{\text{rep}} + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}} + \lambda_{\text{fill}} \mathcal{L}_{\text{fill}}. \quad (9)$$

Here, $\mathcal{L}_{\text{kp}}$ aligns projected template keypoints with annotated image keypoints, and $\mathcal{L}_{\text{rep}}$ re-projects $\mathcal{X}_r$ back to its corresponding image pixels. The silhouette terms utilize a mask distance field to constrain the projected template within the object mask ($\mathcal{L}_{\text{mask}}$) and an occupancy objective to encourage full foreground coverage ($\mathcal{L}_{\text{fill}}$).

Finally, $\mathcal{L}_{\text{reg}}$ penalizes excessive deformation, preserves local geometry through an ARAP-inspired regularizer [55], and discourages assignment collapse.

Altogether, our complete training objective is defined as

$$\mathcal{L} = \lambda_{\text{pair}} \mathcal{L}_{\text{pair}} + \lambda_{\text{geo}} \mathcal{L}_{\text{geo}} + \lambda_{\text{proj}} \mathcal{L}_{\text{proj}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (10)$$

Grasp Transfer and Reconstruction. For a given initial grasp $\mathbf{T}_s^g$ at time s , we recover the corresponding grasp $\mathbf{T}_t^g$ at time t under scene variations. We represent the grasp by two intended contact points on the object and transfer them through the shared template correspondence space. Leveraging soft assignments $\mathbf{A}_s \in \mathbb{R}^{N_s \times M}$ and $\mathbf{A}_t \in \mathbb{R}^{N_t \times M}$, we construct the source-to-target correspondence affinity as

$$\mathbf{C}_{s \rightarrow t} = \mathbf{A}_s \mathbf{A}_t^\top \in \mathbb{R}^{N_s \times N_t}, \quad (11)$$

where $\mathbf{C}_{s \rightarrow t}(i, j) = \langle \mathbf{A}_s(i, :), \mathbf{A}_t(j, :) \rangle$ measures the similarity between source point i and target point j . The shared template enables this construction, while Sinkhorn normalization encourages balanced marginals and reduces many-to-one assignment collapse. Let $\mathbf{c}_{s,1}, \mathbf{c}_{s,2} \in \mathbb{R}^3$ denote the

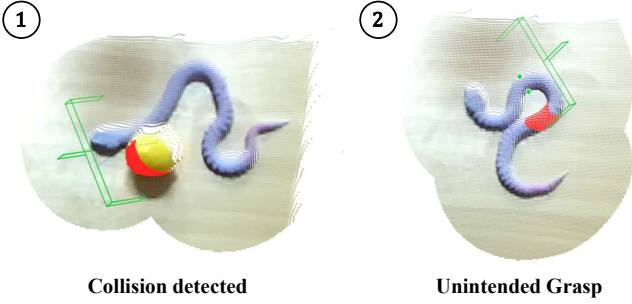


Fig. 3. Illustration of feasibility checking. The green gripper denotes the updated grasp, green points the matched grasp contacts, and red markers infeasible interactions. (1) Collision-prone and (2) unintended grasps are rejected before execution, triggering semantic replanning.

source grasp contacts. Each contact is matched to the highest-scoring target point in the corresponding row of $\mathbf{C}_{s \rightarrow t}$, yielding $\mathbf{c}_{t,1}$ and $\mathbf{c}_{t,2}$.

To reconstruct the grasp pose, the intermediate grasp center $\tilde{\mathbf{p}}_t^g$ and closing direction $\mathbf{d}_t^g$ are computed as:

$$\tilde{\mathbf{p}}_t^g = \frac{\mathbf{c}_{t,1} + \mathbf{c}_{t,2}}{2}, \quad \mathbf{d}_t^g = \frac{\mathbf{c}_{t,2} - \mathbf{c}_{t,1}}{\|\mathbf{c}_{t,2} - \mathbf{c}_{t,1}\|_2}, \quad (12)$$

where a height correction is applied to $\tilde{\mathbf{p}}_t^g$ to obtain the final center $\mathbf{p}_t^g$. Aligning the remaining orientations with the initial grasp constructs $\mathbf{R}_t^g \in \text{SO}(3)$, forming the transferred grasp pose:

$$\mathbf{T}_t^g = \begin{bmatrix} \mathbf{R}_t^g & \mathbf{p}_t^g \\ \mathbf{0}^T & 1 \end{bmatrix} \in \text{SE}(3). \quad (13)$$

The reconstructed grasp and its confidence are then passed to the Information Interaction Module for feasibility checking and subsequent execution or fallback routing.

D. Information Interaction Module

The *Information Interaction Module* bridges semantic grounding, geometric adaptation and execution. It converts semantic outputs into initial grasps, validates geometrically adapted poses, and routes failures back for replanning.

Grasp Initialization. Using the object point cloud and the interaction-grounded local mask, we employ AnyGrasp [56] to generate candidate grasps. Candidates with contacts outside the mask are discarded, and the top-scoring remaining pose becomes the initial $\mathbf{T}_s^g$. Its two surface contacts $\mathbf{c}_{s,1}$ and $\mathbf{c}_{s,2}$ are retained as task-relevant anchors for subsequent geometric adaptation, as detailed in Section III-C.

Feasibility Checking. Each reconstructed grasp is evaluated by its correspondence confidence, reachability, and collision safety. Reachability checking verifies whether the transferred pose is kinematically feasible, while collision checking detects collisions based on the current scene point cloud during gripper approach and closing, including interference with surroundings or unintended parts of the target object. Fig. 3 illustrates two cases. A feasible grasp is directly executed, whereas unreliable correspondence or failed checks trigger fallback to the semantic path for replanning.

IV. EXPERIMENTS

A. Experimental Setup

Hardware. All real-world experiments use a KUKA LBR iiwa 14 R820 arm with a Robotiq 2F-140 gripper. Two ZED 2i cameras provide complementary RGB-D views, one wrist-mounted and one fixed at an elevated viewpoint opposite the robot. Local perception and control modules run on a workstation with an NVIDIA GeForce RTX 3090 GPU.

Evaluation Settings. We evaluate DualManip across six manipulation tasks spanning non-rigid deformation, articulated reconfiguration, rigid motion, and precision assembly, categorized into three general tasks and three PC chassis assembly tasks. The general tasks require the robot to “*put a soft snake into a bowl*” (snake-placement, non-rigid), “*position an Ultraman toy between the monster and astronaut figurines*” (toy-positioning, articulated), and “*place a block fish into a bucket*” (block-placement, rigid). The assembly tasks include RAM Insertion and GPU Insertion, which require inserting a memory module (RAM) and a graphics card (GPU) into fixed chassis slots, respectively, and Fan Placement, which requires placing a cooling fan at its designated mounting location.

We consider three settings: (1) *static*, where the scene remains unchanged throughout execution, providing a baseline measure of system capability; (2) *single-change*, where the object undergoes a single motion or deformation during execution and then remains stationary; and (3) *continuous dynamic*, where the object moves or deforms continuously during execution. The three PC assembly tasks are tested only under the static setting, as they primarily assess precision assembly rather than dynamic adaptation, whereas the general tasks are tested under all three settings. Each object-condition-method combination is evaluated over 15 trials.

Implementation Details. For each object, a reference template is constructed prior to evaluation and kept fixed across all trials. Both the template and observed point clouds are downsampled to 1,024 points for correspondence estimation, i.e., $M = N_t = 1024$. To train the correspondence network, we collect 100 RGB-D observations per object from each camera viewpoint. We set $\lambda_{\text{pair}} = 100$, $(\lambda_{\text{cd}}, \lambda_{3D}) = (10, 100)$, $(\lambda_{\text{kp}}, \lambda_{\text{rep}}, \lambda_{\text{mask}}, \lambda_{\text{fill}}) = (10, 10, 60, 2)$, and $\lambda_{\text{geo}} = \lambda_{\text{proj}} = \lambda_{\text{reg}} = 1$. GPT-5.6 Terra [25] is accessed via the OpenAI API and used as the VLM for semantic reasoning in all experiments, while Qwen3.5-2B [26] is additionally deployed locally for agentic latency evaluation.

B. Baselines

We compare DualManip with four baselines representing distinct responses to scene changes. (1) **ReKep** [15] performs closed-loop execution through sparse keypoint tracking [38]. (2) **OmniManip** [16] represents rigid 6D pose-based adaptation, using object-level pose tracking to update the manipulation target during execution. (3) **CLEA** [18] represents agentic VLM-based adaptation, addressing scene changes through visual validation and high-level replanning rather than direct geometric adaptation. (4) **Open-loop** executes the

TABLE I

QUANTITATIVE RESULTS UNDER THE STATIC SETTING. ENTRIES DENOTE SUCCESSFUL TRIALS, WHILE *Mean* REPORTS AVERAGE SUCCESS RATE (%).

Category	Task	ReKep	OmniManip	CLEA	DualManip (Ours)
General	Snake Placement	8/15	12/15	11/15	12/15
	Toy Positioning	10/15	11/15	12/15	12/15
	Block Placement	9/15	12/15	11/15	11/15
	<i>Mean</i>	60.0%	77.8%	75.6%	77.8%
Assembly	RAM Insertion	1/15	0/15	0/15	4/15
	GPU Insertion	4/15	6/15	5/15	7/15
	Fan Placement	9/15	12/15	11/15	12/15
	<i>Mean</i>	31.1%	40.0%	35.6%	51.1%

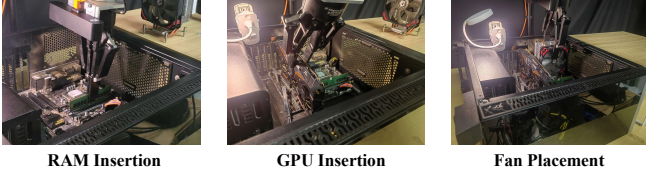


Fig. 4. Representative execution results for the high-precision assembly tasks under the static setting, highlighting DualManip’s ability to perform precise task-conditioned insertion and placement.

initial grasp without geometric adaptation, directly evaluating the impact of omitting scene changes. It is evaluated only under dynamic settings, as no adaptation is needed in static scenes. All methods share the same execution backend where applicable to isolate their scene-adaptation mechanisms.

C. Static Manipulation Results

We first evaluate all methods under the static setting to assess their baseline manipulation capability. Table I shows that DualManip matches the best baseline on general tasks (77.8%) and achieves the highest mean success rate on assembly tasks (51.1%, versus 40.0% for the strongest baseline). Fig. 4 presents the execution results of the assembly tasks. Overall, these results verify that DualManip maintains competitive static manipulation performance before we turn to its primary capability of dynamic adaptation.

D. Dynamic Manipulation Results

We next evaluate adaptation under single-change and continuous dynamic scene variations, as summarized in Table II. Under the single-change setting, DualManip and CLEA both achieve the highest mean success rate of 64.4%, outperforming ReKep (42.2%) and OmniManip (26.7%). OmniManip degrades drastically on the articulated toy and deformable snake, exposing the limitation of object-level rigid pose tracking under articulated and non-rigid changes. ReKep better handles non-rigid motion through sparse key-point tracking, but remains sensitive to unreliable tracks on weakly textured or strongly deforming regions. In contrast, DualManip directly transfers task-relevant contacts through a shared template space, enabling effective and precise adaptation to diverse geometric changes. As shown in Fig. 5, the transferred grasp remains consistent with the original task intent after articulated reconfiguration or rigid displacement. The comparable performance of CLEA indicates that high-level replanning can also recover from an isolated scene

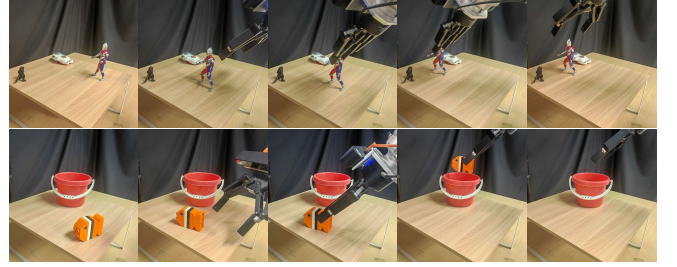


Fig. 5. Representative executions under the single-change setting. Top: toy-positioning under articulated reconfiguration. Bottom: block-placement under rigid displacement.

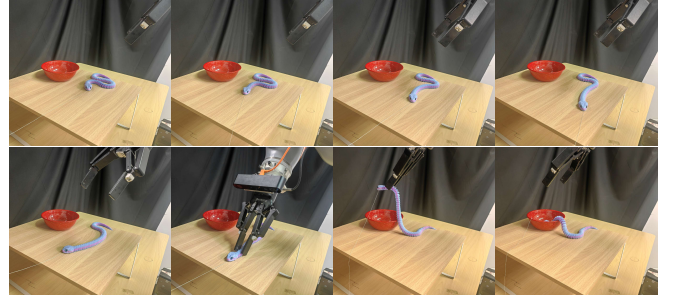


Fig. 6. Qualitative execution on snake-placement under continuous scene variations. DualManip continuously updates grasp poses in real time to accommodate object motion and deformation while maintaining valid task intent.

change, but at considerably higher computational cost, as discussed later.

The advantage of DualManip becomes more pronounced under continuous scene changes. DualManip maintains a 53.3% mean success rate, considerably surpassing ReKep (28.9%) and OmniManip (17.8%), whereas CLEA and Open-loop fail completely. Continuous scene changes further amplify the limitations of sparse point tracking and rigid pose representations, as small geometric mismatches can accumulate throughout execution. CLEA, despite its robust replanning capability, cannot keep pace with continuously evolving geometry due to the latency of repeated high-level reasoning. In contrast, DualManip continuously updates task-relevant contacts via its geometric path, seamlessly adapting execution to evolving geometry while bypassing repetitive VLM inference. Figure 6 showcases this behavior on the deformable snake, where the grasp is continuously adjusted as the object moves and deforms.

TABLE II

QUANTITATIVE RESULTS UNDER DYNAMIC SETTINGS. ENTRIES DENOTE SUCCESSFUL TRIALS, WHILE *Mean* REPORTS AVERAGE SUCCESS RATE (%).

Setting	Task	ReKep	OmniManip	CLEA	Open-loop	DualManip (Ours)
Single-change	Snake Placement	5/15	2/15	9/15	0/15	10/15
	Toy Positioning	7/15	1/15	10/15	0/15	9/15
	Block Placement	7/15	9/15	10/15	0/15	10/15
	<i>Mean</i>	42.2%	26.7%	64.4%	0.0%	64.4%
Continuous	Snake Placement	3/15	1/15	0/15	0/15	8/15
	Toy Positioning	4/15	0/15	0/15	0/15	7/15
	Block Placement	6/15	7/15	0/15	0/15	9/15
	<i>Mean</i>	28.9%	17.8%	0.0%	0.0%	53.3%

TABLE III

QUANTITATIVE COMPARISON OF AVERAGE ADAPTATION LATENCY UNDER A SINGLE SCENE CHANGE.

Method	ReKep	OmniManip	CLEA		DualManip (Ours)
			GPT	Qwen	
Latency	77.2 ms	231.3 ms	6.376 s	4.329 s	138.7 ms

E. Latency Analysis

A central goal of DualManip is to handle frequent geometric changes without repeated high-latency semantic reasoning. We therefore compare adaptation latency, measured from scene update arrival to internal update completion. For DualManip, this includes correspondence estimation and grasp reconstruction, whereas ReKep measures keypoint tracking. Excluding shared downstream modules with large runtime variance, we report the average latency over 20 independent updates for each method.

Table III reports the average adaptation latency of the compared methods under a single scene change. DualManip requires 138.7 ms per adaptation, comparable in latency to ReKep and OmniManip while achieving stronger performance under scene variations. In contrast, CLEA requires 6.376 s for state verification and semantic replanning with GPT [25]. We additionally evaluate CLEA with a locally deployed Qwen3.5-2B [26] model to assess adaptation latency without remote API calls, yet it still has an adaptation latency of 4.329 s, approximately $31\times$ that of DualManip. These results highlight DualManip’s strength in combining low-latency adaptation with robust dynamic manipulation.

V. CONCLUSION

We presented DualManip, a dual-path manipulation framework that decouples high-level semantic reasoning from low-latency geometric adaptation under scene dynamics. By validating shape-adaptive contact transfers prior to execution, DualManip maintains task intent while efficiently adapting to rigid motion, articulated reconfiguration, and non-rigid deformation. Extensive real-world experiments confirm its superior manipulation robustness and markedly lower adaptation latency compared to agentic verification and replanning.

Limitations. DualManip relies on pre-constructed templates and object-specific training for dense correspondence, constraining direct zero-shot generalization to unseen instances. Future work will explore category-level correspon-

dence learning to relax template dependencies while preserving real-time responsiveness.

REFERENCES

- [1] H. Fang, S. Li, S. Wang, X. Xi, D. Liang, and X. Bai, “Towards generalizable robotic manipulation in dynamic environments,” *arXiv preprint arXiv:2603.15620*, 2026.
- [2] H. Xie, B. Wen, J. Zheng, Z. Chen, F. Hong, H. Diao, and Z. Liu, “Dynamicvla: A vision-language-action model for dynamic object manipulation,” *arXiv preprint arXiv:2601.22153*, 2026.
- [3] Y. Lou, H. Gao, X. Zhu, Z. Qiao, X. Han, Y. Yang, Y. Ye, B. Yao, and Z. Pang, “Dynamicwam: Dual-path motion conditioning for world-action models in dynamic manipulation,” *arXiv preprint arXiv:2608.00793*, 2026.
- [4] T. Su, J. Zhu, T. Wang, Y. He, Z. Huang, J. Zhang, C. Ma, H. Wang, T. Zhang, M. Yin, *et al.*, “Demavla: A vision-language-action foundation model for generalizable deformable manipulation,” *arXiv preprint arXiv:2605.31286*, 2026.
- [5] R. Wu, H. Lu, Y. Wang, Y. Wang, and H. Dong, “Unigarmentmanip: A unified framework for category-level garment manipulation via dense visual correspondence,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 16 340–16 350.
- [6] Z. Zhu, Y. Chen, X. Liang, H. Bae, Y. Wang, Z. Yuan, R. Wu, and H. Dong, “Sparse meets dense: Correspondence guided robotic manipulation with rigid-deformable interactions,” *arXiv preprint arXiv:2608.01083*, 2026.
- [7] L. Cui, Z. Zhao, S. Xie, W. Zhang, Z. Han, and Y. Zhu, “Vi-tacman: Articulated object manipulation via vision and touch,” *arXiv preprint arXiv:2510.06339*, 2025.
- [8] J. Zhu, J. Zhang, T. Su, T. Liu, Z. Wang, K. Xie, Z. Huang, C. Ma, Y. He, T. Wang, *et al.*, “Dswam: A dual-system world action foundation model for fine-grained robot manipulation,” *arXiv preprint arXiv:2607.04927*, 2026.
- [9] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “Voxposer: Composable 3d value maps for robotic manipulation with language models,” *arXiv preprint arXiv:2307.05973*, 2023.
- [10] H. Huang, F. Lin, Y. Hu, S. Wang, and Y. Gao, “Copa: General robotic manipulation through spatial constraints of parts with foundation models,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 9488–9495.
- [11] W. Tang, J.-H. Pan, Y.-H. Liu, M. Tomizuka, L. E. Li, C.-W. Fu, and M. Ding, “Geomanip: Geometric constraints as general interfaces for robot manipulation,” *arXiv preprint arXiv:2501.09783*, 2025.
- [12] W. Tang, J. Gao, J.-H. Pan, G. Wang, L. E. Li, Y.-H. Liu, M. Ding, P.-A. Heng, and C.-W. Fu, “Rethinking intermediate representation for vlm-based robot manipulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 29 652–29 662.
- [13] W. Shen, N. Kumar, S. Chintalapudi, J. Wang, C. Watson, E. Hu, J. Cao, D. Jayaraman, L. P. Kaelbling, and T. Lozano-Pérez, “Tiptop: A modular open-vocabulary planning system for robotic manipulation,” *arXiv preprint arXiv:2603.09971*, 2026.
- [14] H. Liu, Y. Xue, Y. Zhou, H. Deng, Y. Liang, L. Xie, and Z. Wang, “Unimanip: General-purpose zero-shot robotic manipulation with agentic operational graph,” *arXiv preprint arXiv:2602.13086*, 2026.
- [15] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” *arXiv preprint arXiv:2409.01652*, 2024.

- [16] M. Pan, J. Zhang, T. Wu, Y. Zhao, W. Gao, and H. Dong, "Omnimanip: Towards general robotic manipulation via object-centric interaction primitives as spatial constraints," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 17 359–17 369.
- [17] H.-H. Nguyen, M. N. Vu, F. Beck, G. Ebmer, A. Nguyen, W. Kemmetmüller, and A. Kugi, "Language-driven closed-loop grasping with model-predictive trajectory optimization," *Mechatronics*, vol. 109, p. 103335, 2025.
- [18] M. Lei, G. Wang, Y. Zhao, Z. Mai, Q. Zhao, Y. Guo, Z. Li, S. Cui, Y. Han, and J. Ren, "Clea: Closed-loop embodied agent for enhancing task execution in dynamic environments," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 21 048–21 054.
- [19] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choro-manski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [20] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "Open-vla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [21] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, " $\pi_{0.5}$: a vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.
- [22] P. Zhou, S. Chen, D. Chen, J. Wang, R. Jin, B. Zhu, Y. Pan, S. Gu, K. Wang, S. Nan, *et al.*, " π_0 -WM: A unified video-action world model for robotic manipulation," *arXiv preprint arXiv:2606.01027*, 2026.
- [23] Y. Liu, P. Sun, S. Li, Y. Xie, L. Zhang, X. Chao, S. Dong, F. Chen, X.-P. Zhang, and W. Ding, "Oa-wam: Object-addressable world action model for robust robot manipulation," *arXiv preprint arXiv:2605.06481*, 2026.
- [24] R. Chen, Y. Yang, Z. Tang, D. Huo, T. Lin, H. Wu, H. Liu, Y. Chen, L. Zheng, B. Yuan, *et al.*, "Abot-m0. 5: Unified mobility-and-manipulation world action model," *arXiv preprint arXiv:2607.00678*, 2026.
- [25] OpenAI, "GPT-5.6 System Card," <https://deploymentsafety.openai.com/gpt-5-6>, 2026.
- [26] Qwen Team, "Qwen3.5: Towards native multimodal agents," February 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [27] X. An, Y. Xie, F. Tang, Y. Yan, H. Tan, D. Zhu, C. Chen, X. Zhao, B. Qin, K. Yang, *et al.*, "Llava-onevision-2: Towards next-generation perceptual intelligence," *arXiv preprint arXiv:2605.25979*, 2026.
- [28] F. Liu, K. Fang, P. Abbeel, and S. Levine, "Moka: Open-vocabulary robotic manipulation through mark-based visual prompting," in *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.
- [29] M. Sun, J. Wei, Q. He, D. Wang, C. Lu, and J. Sun, "Executable analytic concepts as the missing link between vlm insight and precise manipulation," *arXiv preprint arXiv:2510.07975*, 2025.
- [30] S. Xu, R. Jin, H. Zhou, B. Yue, G. Qiao, Y. Tai, Y. Deng, K. Jia, and G. Liu, "From reaction to anticipation: Proactive failure recovery through agentic task graph for robotic manipulation," *arXiv preprint arXiv:2605.11951*, 2026.
- [31] S. He, S. Wang, B. Yue, J. Teng, C. Wang, and G. Liu, "Roboreact: Agentic skill distillation from generated egocentric videos for generalizable whole-body manipulation," *arXiv preprint arXiv:2608.03387*, 2026.
- [32] Z. Liu, X. Ning, Z. Hu, X. Xie, W. Li, Z. Tang, C. Wang, Z. Yang, H. Wang, Y. Liu, *et al.*, "Goal2skill: Long-horizon manipulation with adaptive planning and reflection," *arXiv preprint arXiv:2604.13942*, 2026.
- [33] Z. Lei, C. Liu, Y. Xiong, M. Xiong, Y. Ding, Z. Zhang, W. Li, and S. Chen, "Towards long-horizon embodied agents with tool-aligned vision-language-action models," *arXiv preprint arXiv:2605.13119*, 2026.
- [34] Y. Zhang, H. Zhang, F. Gao, X. Li, Z. Liu, C. Zhu, J. Qiu, Y. Yan, J. Liu, W. Tang, *et al.*, "Harness vla: Steering frozen vlans into reliable manipulation primitives via memory-guided agents," *arXiv preprint arXiv:2607.08448*, 2026.
- [35] B. Wen, C. Mitash, B. Ren, and K. E. Bekris, "se (3)-tracknet: Data-driven 6d pose tracking by calibrating image residuals in synthetic domains," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 10 367–10 373.
- [36] B. Wen, W. Yang, J. Kautz, and S. Birchfield, "Foundationpose: Unified 6d pose estimation and tracking of novel objects," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 17 868–17 879.
- [37] A. W. Harley, Z. Fang, and K. Fragkiadaki, "Particle video revisited: Tracking through occlusions using point trajectories," in *European Conference on Computer Vision*. Springer, 2022, pp. 59–75.
- [38] N. Karaev, Y. Makarov, J. Wang, N. Neverova, A. Vedaldi, and C. Rupprecht, "Cotracker3: Simpler and better point tracking by pseudo-labeling real videos," in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2025, pp. 1–10.
- [39] Y. Xiao, Q. Wang, S. Zhang, N. Xue, S. Peng, Y. Shen, and X. Zhou, "Spatialtracker: Tracking any 2d pixels in 3d space," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 20 406–20 417.
- [40] T. Groueix, M. Fisher, V. G. Kim, B. C. Russell, and M. Aubry, "3d-coded: 3d correspondences by deep deformation," in *European Conference on Computer Vision*. Springer, 2018, pp. 235–251.
- [41] J. Lei and K. Daniilidis, "Cadex: Learning canonical deformation coordinate space for dynamic surface representation via neural homeomorphism," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 6614–6624.
- [42] Y. Li and T. Harada, "Lepard: Learning partial point cloud matching in rigid and deformable scenes," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 5544–5554.
- [43] Z. Chen, P. Jiang, and R. Huang, "Dv-matcher: Deformation-based non-rigid point cloud matching guided by pre-trained visual features," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 27 264–27 274.
- [44] Z. Teed and J. Deng, "Raft: Recurrent all-pairs field transforms for optical flow," in *European conference on computer vision*. Springer, 2020, pp. 402–419.
- [45] H. Xu, J. Zhang, J. Cai, H. Rezatofighi, and D. Tao, "Gmflow: Learning optical flow via global matching," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 8111–8120.
- [46] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris Coll-Vinent, C. Ryal, K. V. Alwala, H. Khedr, A. Huang, *et al.*, "Sam 3: Segment anything with concepts," in *International conference on learning representations*, vol. 2026, 2026, pp. 138 846–138 923.
- [47] Z. Qi, W. Zhang, Y. Ding, R. Dong, X. Yu, J. Li, L. Xu, B. Li, X. He, G. Fan, *et al.*, "Sofar: Language-grounded orientation bridges spatial reasoning and object manipulation," *Advances in neural information processing systems*, vol. 38, pp. 76 367–76 412, 2026.
- [48] Z. Lai, Y. Zhao, H. Liu, Z. Zhao, Q. Lin, H. Shi, X. Yang, M. Yang, S. Yang, Y. Feng, *et al.*, "Hunyuan3d 2.5: Towards high-fidelity 3d assets generation with ultimate details," *arXiv preprint arXiv:2506.16504*, 2025.
- [49] M. Tian, M. H. Ang Jr, and G. H. Lee, "Shape prior deformation for categorical 6d object pose and size estimation," in *European Conference on Computer Vision*. Springer, 2020, pp. 530–546.
- [50] K. Chen and Q. Dou, "Sgpa: Structure-guided prior adaptation for category-level 6d object pose estimation," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2021, pp. 2753–2762.
- [51] M. Cuturi, "Sinkhorn distances: Lightspeed computation of optimal transport," *Advances in neural information processing systems*, vol. 26, 2013.
- [52] C. Cuttano, G. Trivigno, C. Masone, and S. Roth, "Marco: Navigating the unseen space of semantic correspondence," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 21 649–21 658.
- [53] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1440–1448.
- [54] H. Fan, H. Su, and L. J. Guibas, "A point set generation network for 3d object reconstruction from a single image," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 605–613.
- [55] O. Sorkine, M. Alexa, *et al.*, "As-rigid-as-possible surface modeling," in *Symposium on Geometry processing*, vol. 4, 2007, pp. 109–116.
- [56] H.-S. Fang, C. Wang, H. Fang, M. Gou, J. Liu, H. Yan, W. Liu, Y. Xie, and C. Lu, "Anygrasp: Robust and efficient grasp perception in spatial and temporal domains," *IEEE Transactions on Robotics*, vol. 39, no. 5, pp. 3929–3945, 2023.